

# FRED TALKS

25<sup>th</sup> April 2026

---

## CONTENTS

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

---

[Daily article] April 25: Bombing of Obersalzberg

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 323 WORDS

---

BugBash'26 Keynote

NOREPLY@BLOGGER.COM (MURAT) · 1307 WORDS

---

You should try contra dancing

BEN KUHN · 2470 WORDS

---

2.1.120

CHANGELOG · 384 WORDS

---

# Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

---

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

## Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

### Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the  $M \times N$  integration problem.

### Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

## Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

## Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

## Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

### Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

## Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.

## Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (`search` and `execute`) cover ~2,500 endpoints in roughly 1K tokens.

## Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

[Elicitation](#) lets your server pause mid-tool call to ask the user for input. [Form mode](#) sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. [URL mode](#) hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

## Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest [MCP spec](#) supports [CIMD](#) (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. [Vaults in Claude Managed Agents](#) covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

## Making MCP clients more context-efficient

MCP standardizes how AI agents ([clients](#)) connect to and work with tools and data sources they need ([servers](#)). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

### Load tool definitions on demand with tool search

[Tool search](#) defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our [testing](#), tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: [advanced tool use](#)

### Process tool results in code with programmatic tool calling

[Programmatic tool calling](#) processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly [37%](#) on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See [advanced tool use](#) for the full breakdown.

# Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

## Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our data plugin for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: [Extending Claude's capabilities with skills and MCP servers](#)

## Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. Canva, Notion, Sentry, and more do this today in Claude, listing the skill next to their connector in our web directory.

To make that pairing portable across every client, the MCP community is actively working on an extension for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

## The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

## Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

---

# [Daily article] April 25: Bombing of Obersalzberg

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 25 APR 2026 · [SOURCE](#)

---

The bombing of Obersalzberg was a World War II air raid that targeted a complex of residences and bunkers built for Adolf Hitler at Obersalzberg, Bavaria. It was carried out by 359 Allied heavy bombers on 25 April 1945, during the last days of World War II in Europe. The Allies believed that the German government would use the Obersalzberg complex to command an Alpine Fortress. Many buildings were destroyed, though the Berghof, which was Hitler's residence, and the bunker network were only slightly damaged. Hitler was not present. Two bombers were shot down with the loss of four airmen, and 31 Germans were killed. Historians have identified several motives for the attack, including supporting Allied ground forces, demonstrating the effectiveness of the heavy bomber, convincing die-hard Germans that the war was lost and obscuring the memory of pre-war appeasement policies. The raid was celebrated at the time, but as the Alpine Fortress proved to be a myth most post-war histories make little mention of it. (Full article...).

Read more: [https://en.wikipedia.org/wiki/Bombing\\_of\\_Obersalzberg](https://en.wikipedia.org/wiki/Bombing_of_Obersalzberg)

---

Today's selected anniversaries:

1644:

Ming–Qing transition: The Ming dynasty of China fell when the Chongzhen Emperor committed suicide during a peasant rebellion led by Li Zicheng. [https://en.wikipedia.org/wiki/Ming\\_dynasty](https://en.wikipedia.org/wiki/Ming_dynasty)

1920:

A march by blind people from across the United Kingdom to protest poor working conditions ended at Trafalgar Square, London. [https://en.wikipedia.org/wiki/1920\\_blind\\_march](https://en.wikipedia.org/wiki/1920_blind_march)

1986:

Mswati III was crowned King of Swaziland, succeeding his father Sobhuza II. [https://en.wikipedia.org/wiki/Mswati\\_III](https://en.wikipedia.org/wiki/Mswati_III)

2014:

A water crisis began when officials in Flint, Michigan, switched the city's water supply to the Flint River, leading to contamination. [https://en.wikipedia.org/wiki/Flint\\_water\\_crisis](https://en.wikipedia.org/wiki/Flint_water_crisis)

---

Wiktionary's word of the day:

skerrick: 1. (originally UK, dialectal, now chiefly Australia, New Zealand, chiefly in the negative) A very small amount or portion; the least bit. 2. About Word of the Day 3. Nominate a word 4. Leave feedback <https://en.wiktionary.org/wiki/skerrick>

---

Wikiquote quote of the day:

” --Wolfgang Pauli [https://en.wikiquote.org/wiki/Wolfgang\\_Pauli](https://en.wikiquote.org/wiki/Wolfgang_Pauli)

---

## BugBash'26 Keynote

NOREPLY@BLOGGER.COM (MURAT) · 25 APR 2026 · [SOURCE](#)

---



I attended [the BugBash 2026](#) these last two days, and had a blast. Here are my notes from the first keynote. I will try to find time to publish my notes from the other talks in the coming days.

# Keynote: We won, what now?

Will Wilson, Co-founder & CEO @ Antithesis

The Antithesis team opened with a great animation/teaser clip, then Will took the stage. Here is the summary of his talk.

This is not a software testing conference. This is about building reliable software by any means: testing, observability, formal methods, people/culture, better languages. He shows a meme of fantastic five or something using their rings, they invoke this giant warrior.

Time to acknowledge the elephant in the room. A new contender emerges: AI!!

We are now taking a fundamentally unreliable system (AI) to make the systems we are developing reliable. And it is working somehow?! There is a vibe quality to it. When the cost of software generation goes down drastically, you can do a whole lot of it as per Jevons' paradox.

At this point Will starts talking about this hypothetical band: **Quaternion dysfunction**, and its noncommutative album. This is a niche band, following them early on makes us feel very special, part of a small in-group.

Now imagine that this niche band becomes freakishly popular suddenly. You feel many things. First you feel a great validation. But now more people start following the band, and you lost the in-group identity. You need to get a new personality. But who knows when maybe you can cash on it, as expert or talent lead.

Other copycat bands enter the scene, say Helvetica scenario. And now there are also many faker fans. People just follow these bands because they are popular. This happens in real world a lot, and in the technology world as well. Jon Evans wrote a Techcrunch article on this in 2015: [Beware the pretty people](#). Lawyers, financiers, business people. He overstates the effects. Silicon Valley getting popular was mostly good diversity with these other people arriving. But yes there are also comes scammers, bad actors.

Here is a comparison of niche fields versus popular fields.

- elite vs. energizing
- elitist vs. lots of BS

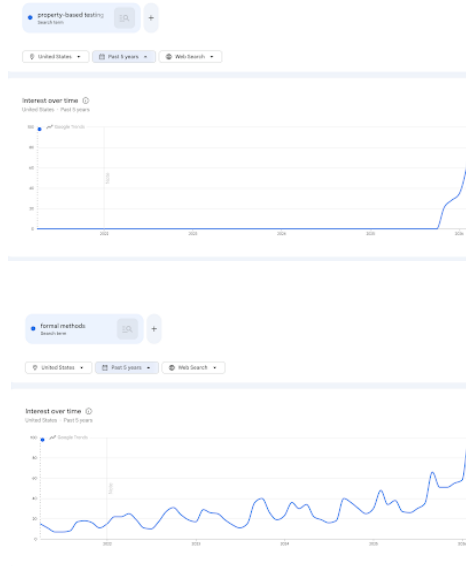
- cozy vs. innovative
- defiant vs. ridiculous

Transition between these two worlds can be traumatic. And when all is said and done, Michael Lewis will come and write a book about it. (Will's zinger, not mine!)

In case, you still haven't caught on to the analogy. That band is "software correctness": formal methods, property-based testing, observability. I.e., the people/community inc BugBash.

As Will put it bluntly, you belong to a cult. Vast majority of engineers don't care about correctness much. They are not bad people, but they are doing this because other pressures/priorities. This is a fact of life: most people don't didn't care about correctness much.

Well, that is until something strange happened, which made people care about correctness! Check the Google Trends for property based testing. It shot up from zero to millions in 2025-26. Same for formal methods.



Everybody just started caring about this because of AI. But how has AI caused this to happen? The conventional story is that AI agents don't write correct software, but take this story at face value. You mean software is written by unreliable agents? Always has been meme! People have been writing bad software for decades, and nobody batted an eyelash before for verification. So why now all of a sudden they care about verification.

The Amdahl's law is behind it. Focus on the part that is slow, the bottleneck, for improvement.

Previously when Will told to the managers that 50% of your teams time is spent in testing, they didn't use to believe him. Now they correct him and tell him it is 99%. Implication of AI and Amdahl's law means, now correctness is important. So no need to mention that, thanks to the AI wave, business is booming for Antithesis.

The Amdahl's law is a nice angle to look at this. But I think there is another reason for this, as Steve Klabnik mentioned in day 2. Previously, no matter how buggy it is, you had written your software, understood it, and tried it. And using AI breaks all three: now you don't have a way to validate the software without formal methods and property-based testing, etc.

Then Will went on to set up the roadmap and expectations for the software reliability folks.

This feels like the Eternal September (1993/1994), where the unwashed masses started onboarding the internet. Forums got flooded, the norms changed. The in-crowd protested, but it was for good. It was an overall positive. We should keep the looking back perspective in mind.

What about the payoffs? Will showed the Rembrandt painting titled the "Parable of the Workers in the Vineyard", which depicts the bible story of vineyard workers getting paid at the end of the day, where the workers who joined in the last couple hours of the day paid the same as the ones who toiled all day. The parable is interpreted to mean that even those who are converted late in life earn equal rewards along with those converted early, and that people who convert early in life need not feel jealous of those later converts.

Will iterated: Don't feel resentful. This is what winning looks like. Other people coming and coopting your thing is actually what winning looks like. It is okay to win! Your position will get demolished/bastardized, but the world would have moved slightly towards your position. This is the transition from defiant to ridiculous in the above table.

(My aside: As for one, I am tired of winning! Too much winning going on on all fronts recently. I feel like the word "winning" is getting devalued. Also I personally do not agree with the parable's lesson. Even the monkey's have this injustice instinct built in. Don't go philosophizing over me.)

Anyway, Will's takeaway message is this. The masses are coming. It is our community's time to shine. Software reliability tools had been for the elite, but it is changing. It is time to **teach others.**

Teach others?! On day two, Steve Klabnik also iterated this message. It is time for others to learn from this community. But, neither elaborated how this teaching/learning will take place. And I remain skeptical. Yeah, I do blog about this stuff, and enthusiasts and people in the know follow and they say they benefit and learn. But I am skeptical about how this would scale. Learning is an active process, it requires active participation and effort on the learner's side. Some educators even claim, there is no teaching, there is only learning. I am worried people will follow easy non-solution trends, like I don't know HOPE: Heuristic Oversight of Probabilistically-correct Execution. Or I don't know AGILE: Assert Goodness, Iterate Later, Eventually. The braindead solutions always get more popular. Thinking is hard, and the human brains are optimized to be lazy.

Let me talk about the talk mechanics to wrap this up. Overall, this was a good show, in the best sense of the word. The delivery of the talk looked effortless but it is clear Will put a lot of work in to this presentation to make it this smooth. He had so many zingers, and in-jokes. The band analogy is wonderful. The Rembrands painting story is really memorable. These set the stage well, and help people manage expectations for the roadmap. This is a technical talk, presented as a nontechnical talk.

It was very entertaining, as well as informative and thought-provoking. Will's liberal arts background comes through clearly. And the clever use memes was also a pattern shared among the best presenters in the conference. For a conference like this, the point is to score laughs, and entertain as much as teach.

---

## You should try contra dancing

BEN KUHN · 25 APR 2026 · [SOURCE](#)

---

A popular icebreaker in San Francisco these days is “How would you spend your life if AGI meant nobody needed to work?” For me, I think a surprisingly big part of the answer is a dorky-sounding kind of folk dance called contra dancing.

I started trying to answer that question by thinking: what are the things I do *atelically*—because I enjoy them for their own sake, not in pursuit of some longer-term goal? For me, a lot of that has to do with things that are (1) physically joyful, and (2) help me feel connected to other people.

My pitch for contra dancing is that among physically joyful and connecting activities, it's one of the ones where it's *easiest to get to the point where it's fun*, and therefore one of the best to start out with. Many other forms of dance require weeks of lessons before you're even encouraged to do it socially at all. By contrast, the format of contra dancing means that you can go to a half-hour beginner lesson before your first dance, and there's a good chance you will be having a lot of fun by the end of the evening.

What's more, the fact that it's so easy to start having fun shapes its community and priorities, such that it has among the best vibes of any activity I do.

---

I first encountered contra in middle school. My mom ran an intentional community, and we had a housemate move in named Bree. Bree was *obsessed* with this thing called “contra dancing,” and every week she would try to convince me and our other housemates to come with her.

Which I absolutely refused. I was an extremely uncool middle schooler, and that meant I was terrified of doing anything that even whiffed of uncoolness, lest I give my classmates another reason to judge me. I was also deeply awkward; I'd never even been to a school dance and barely talked to girls. But Bree was persistent, and one of our other housemates got hooked, and eventually I was intrigued.

By then, though, I was dug in as a contra hater, and I was scared that if I admitted I had changed my mind, my housemates would make fun of me. So I kept refusing.

Finally, my housemate Thomas, who was even more of a skeptic than I was, told Bree that he would only go if I went, presumably so that she would stop bugging him and bug me instead. Aha—my golden opportunity! I could pass off my change of heart as a prank on Thomas. “Okay, sure! It's a deal!”

Bree and I showed up at the Concord Scout House one Thursday evening and jumped into the next dance. It was an unusually complex square dance, and I got confused—in fact, so confused that my entire square ended up giving up halfway through.<sup>1</sup>

I was mortified. Not only was I doing a deeply uncool activity, I was *the least cool person there*, ruining it for all the slightly less uncool people with my incompetence and lameness.

Except that the other dancers didn't seem to see it that way. Instead of resenting me or making fun of me, they said things like “Gosh, that one was really tricky!” and “Sorry, that's really not what it's usually like!” and “Please don't give up, try a few more dances! Want to dance with me?”

I tried some more dances, I didn't cause any more collapses, and by the end of the night I was overflowing with exhilaration and joy. I went back the next week, and the week after that. I went to the twelve-hour [Snow Ball](#) and then the weekend-long [Flurry Festival](#). I got to feel cool for the first time in my life when people started being excited to dance with me, and even cooler when some of them asked me on dates. It was a huge part of my life for a while, and although I'm now dancing at maybe a quarter of my peak intensity, it's still one of my most joyful and treasured things I do.

This happens to about one in ten people I take contra dancing: something clicks and it becomes a big part of their life.

---

So what exactly *is* contra dancing?

When people ask me this I often feel stuck, because describing the mechanics of a contra dance doesn't really capture its essence or why it's fun, but without understanding the mechanics, the essence doesn't really make sense. So I'll try both. But you have to promise to read through to the essence part, and not just bounce off if the mechanics sound silly.

Mechanically: you show up to the dance hall. You ask someone to dance, or they ask you. You join a long line ("set") of other couples and take hands in a four-person group with one adjacent ("neighbor") couple. A *caller* stands at the top of the dance hall and does a "walk-through" of the dance you're about to do, calling out a series of figures ("circle left three places," "do-si-do your neighbor," etc.) At the end, you've moved up or down the line, and you're ready to do the same thing with some new neighbors coming at you.

Next to the caller is a live band. (They probably sound a bit like [this](#).) The pianist plays [four potatoes](#) and the fiddler launches into a reel. The caller calls out the figures for the first few times through the dance, then drops out as the dancers get into flow.

It looks a bit like this video, although I don't know how much this will transmit the vibe, since contra is a lot more fun to do than it is to watch:

Bonus: you can see high school me in this dance! I'm in the dark red T-shirt at the very right of the frame at the beginning.

---

To me, the *essence* of contra dancing is about flow, joy, and community.

You can't really tell from watching, but a lot of what makes for good contra dancing is learning how to trade momentum back and forth ("share weight") with other dancers so that each figure flows seamlessly into the next one. When you get this right, it feels amazing. Subtle changes in how your partner holds your hand cue you into the next figure. The dance becomes effortless. You can turn your brain off and just do what the giant dancing super-organism invites you to do.

It's also physically joyful. The core figure of contra dancing is called the *swing*, in which you and your partner take a waltz-like hold and pivot around a shared center. Done right, it feels like a cross between flying and a world-class hug. Even outside of the swing, the shared weight, smooth momentum, and twirling "flourishes" will make you feel weightless, graceful and deeply connected to the music and the other dancers.

Lots of kinds of dance feel flowy and exhilarating when done well. What makes contra different is how easy it is to get there. Most forms of dance require weeks of lessons before they get fun. In contra, all you need to do is show up for one beginner lesson half an hour before the dance starts.<sup>2</sup> Of course, there's plenty to learn after that—improving your swing technique, learning how to share weight, and where to put flourishes for the best joy and flow—but all of that is for getting you from a 8/10 to an 11/10 on the fun scale; getting to an 8 is pretty straightforward.

So contra makes it easy to have a baseline level of fun dancing with anyone. And because it's a group dance, you end up feeling connected to your entire set, not just your partner. I think those two things are behind my other favorite part of contra dancing, which is how friendly and welcoming the community is.

Many kinds of dance are a bit snooty—the experienced dancers look down on newbies and try to avoid dancing with them. By contrast, my experience at my first dance, where I caused my square to crash and burn and was met with apologies and "I hope you'll give it another shot," is typical for contra. A lot of contras explicitly ask experienced dancers to dance with newcomers. It's just hard to be snooty when you're having so much fun! You want to share it with everyone.

3

As draft reader Chloe put it: "I think in many ways, your story of your first dance is the heart of it—you can't do it wrong. Or even if you do, it somehow leads to more love."

---

In my opinion, this makes contra dancing *one of the best entry points to dance*:

- Each dance is walked through, so you don't have to worry about deciding what to do.
- You can get to an 8/10 fun level, where practice becomes rewarding instead of a chore, really quickly.

- The community is incredibly welcoming, which makes it a lot less stressful to be a beginner.

While it's structured enough to be beginner-friendly, there's also enough room for improvisation and technique to give it a lot of depth. This makes it a great on-ramp to other dances and movement practices. When I started contra dancing I had a lot of fear and blocks about improvised dancing (for example, I didn't feel comfortable dancing at standard American high school/wedding style dances), but recently I tried out fusion and contact improv and had fun at both. It's pretty common for people to use contra as a gateway into more difficult forms of dance.

Most importantly, though, it's just incredibly joyful on its own! It's the only activity I know of where I've had multiple friends mention that every time they go, their cheeks hurt from smiling so much.

---

Beyond just fun, multiple draft readers independently commented that contra dance is a precise opposite of many things that bother them about... the rest of life:

- Much of my life happens on a screen in motionless silence; contra is as three-dimensional, movement- and sound-filled as you can get.
- The default mood of a lot of spaces is a sort of ironic detachment.<sup>4</sup> The default mood of contra dancing is earnest joy.
- In everyday life it's easy to end up atomized and isolated. At a contra, everyone does the same steps and makes the dance together; it's deeply communal.
- In a lot of social contexts (especially in San Francisco), I often feel like I'm being "sized up" for how interesting or important I am. At most contras, people are just happy I'm there.
- Many things I do (work, choral music, blogging...) reward excellence above almost everything. Contra welcomes imperfection: we'd rather have an amateur live band than recorded professionals, and don't believe in more than half an hour of organized teaching.

This makes it a grounding and nourishing antidote to a lot of everyday stressors. When I was so drawn to it in middle school, I don't think it was just about the fun: on some deep level it was very good for me.

---



If I've gotten you excited enough to try contra, here's what you should do:

- Go to [try contra dot com](https://trycontra.com) to find dances near you. If there are multiple dances, I recommend trying to figure out which one is the largest (larger dances tend to have more energy and better vibes), although that might be tricky to figure out from the websites.
- Make a note of where and when the dance is, including the beginner lesson. Remember to show up beforehand for that!
- Dress code wise: contra dancing is generally light cardio; wear something you are happy to be active in.<sup>5</sup>
  - Most people dress pretty casually; many women (and some men!) wear skirts, especially skirts that do fun things when you twirl.
  - Look at the dance site to see if they say anything about footwear; some dances request that you wear soft-soled shoes with clean soles to avoid damaging floors.
  - Bring a water bottle! There are fountains at most dances but it helps to be able to hydrate more quickly since the breaks between dances are short.
- If you go to a dance and like it, the easiest way to go from 8/10 fun to 9.5/10 is to find someone with a really good swing and ask them how you can improve your technique. (If you're not sure, look for someone who's fast and smooth.) They will be tickled that you asked!<sup>6</sup>
- If you have a friend who contra dances, try going with them! They will be thrilled to bring you and it'll be even more fun for both of you.<sup>7</sup>

Have fun! I hope you smile so much your cheeks hurt.

---

*Thanks to Jeff Kaufman, Alex Allain, Chloe Lubinski, and Jessie Brown for commenting on a draft of this post.*

*Further reading/listening/other consumption:*

- [My playlist of favorite contra dance music](#) (because every dance has live music, contra has a thriving music scene that punches far above its weight!)
- Jeff Kaufman, [Contra Dance as a Model For Post-AI Culture](#)
- [try contra dot com](https://trycontra.com)

- 
1. This is at least a 1-in-10,000 event; I've never seen it happen again in over 20 years of regular dancing.
  2. In fact, you can probably get away without even that!
  3. Another benefit of this is that contra is much more intergenerational than most forms of dance. I have dance friends in their twenties and seventies!
  4. At least for me, when I overuse ironic detachment it has subtle but far-reaching effects on how I see and engage with the world. It's an easy way to amuse people, but ends up masking or numbing out a lot of more "real" responses. I've found it really interesting and a bit psychoactive to play what I call "the ironic detachment game," in which I am banned from responding to anything in an ironically detached way.
  5. I personally sweat a lot and usually bring multiple shirts to change into, although most other dancers I know don't need to do this.
  6. Teaching a swing is a separate skill from just having a good swing, so you may want to ask a few different people.
  7. I recently started a mission to get my friends into contra dancing (hence also this post)! It's going shockingly well and it's made it way more fun for me!
- 

## 2.1.120

CHANGELOG · 25 APR 2026 · [SOURCE](#)

---

- Windows: Git for Windows (Git Bash) is no longer required — when absent, Claude Code uses PowerShell as the shell tool
- Added `claude ultrareview [target]` subcommand to run `/ultrareview` non-interactively from CI or scripts — prints findings to stdout (`--json` for raw output) and exits 0 on completion or 1 on failure
- Skills can now reference the current effort level with `${CLAUDE_EFFORT}` in their content

- Set `AI_AGENT` environment variable for subprocesses so `gh` can attribute traffic to Claude Code
- Spinner tips that recommend installing the desktop app or creating skills/agents are now hidden when you already have them
- Show a "use PgUp/PgDn to scroll" hint when the terminal sends arrow keys instead of scroll events
- Faster session start when you have many `claude.ai` connectors configured but not authorized
- The auto mode denial message now links to the configuration docs
- `claude plugin validate` now accepts `$schema`, `version`, and `description` at the top level of `marketplace.json` and `$schema` in `plugin.json`
- Auto-compact in auto mode now displays `auto` (lowercase, no token count) instead of a misleading token value
- Fixed pressing Esc during a stdio MCP tool call closing the entire server connection (regression in 2.1.105)
- Fixed `/rewind` and other interactive overlays not responding to keyboard input after launching with `claude --resume`
- Fixed terminal scrollbar duplication in non-fullscreen mode (resize, dialog dismiss, long sessions)
- Fixed `DISABLE_TELEMETRY` / `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC` not suppressing usage metrics telemetry for API and enterprise users
- Fixed false-positive "Dangerous rm operation" permission prompts in auto mode for multi-line bash commands containing both a pipe and a redirect
- Fixed long selection menus clipping below the terminal in fullscreen mode — the focused option now stays on screen as you scroll
- Fixed Write tool output collapsing instead of expanding when clicking "+N lines" in fullscreen

- Fixed slash command picker jumping while typing, and improved highlight to only match contiguous substrings in blue
- Fixed `/plugin` marketplace failing to load when one entry uses an unrecognized source format — that entry is shown but installing it prompts you to update
- [VSCode] `/usage` now opens the native Account & Usage dialog instead of returning plain-text session cost
- [VSCode] Voice dictation now respects the `language` setting in `~/.claude/settings.json`
- Fixed `find` in the Bash tool exhausting open file descriptors on large directory trees, causing host-wide crashes (macOS/Linux native builds)